

Take-Home Assignment — Case Review for Fraud Investigators

Estimated effort: a couple focused evenings (we are explicitly not asking you to build production software). We care more about your thinking, your product instincts, and the working slice you can demonstrate — not feature count.

1. The Context

A large insurance company runs a fraud investigation team. Every day, a rules + ML "fraud engine" produces a queue of suspicious claims (referrals) that get routed to human investigators. Today the investigators:

- Open a legacy reactive tool, paste a claim number, and pull data manually.
- Read through dozens of signals per case to figure out whether the referral is real, a false positive, or needs deeper digging.
- Repeat this for every case in their queue. It is slow, inconsistent across investigators, and burns the most time on the cases that turn out to be benign.

We want to flip the model from **reactive** to **proactive**:

"When an investigator opens the tool in the morning, they should already see an AI-generated triage of their queue — which cases look low-risk / likely false positive, which look genuinely suspicious — along with the reasoning, the evidence, and a recommended next step. The human stays in control, but the AI does the heavy lifting first."

That is the product you are being asked to design and prototype.

2. Your Mission

Design and build a **working prototype** of an **AI-First Case Review** tool — what we are loosely calling a "*Junior AI Investigator*."

The goal is to demonstrate, end-to-end (even if narrow), how an AI system can:

1. Ingest a queue of referred claims.
2. Reason over each case (signals, claim attributes, anything else useful).
3. Produce a **per-case assessment** before a human looks at it: a short summary, the key indicators that drove the conclusion, a risk level, and a recommended next action.
4. Let an investigator interact with the case — drill in, ask follow-up questions in natural language, accept/reject the AI's findings, add notes, and move the case forward.

How you actually build it — architecture, tech stack, models, agent design, UX — **is up to you**. We want to see how *you* would approach this problem.

3. What We're Giving You

A single CSV file: [sample_cases.csv](#)

- **50 synthetic claim records.** No real customer or provider data — everything is fabricated.
- A handful of basic case attributes (claim id, date, care type, amount, state).
- **9 fraud-style signals** per case. They are intentionally generic, common-sense indicators you would expect to see in any fraud-detection setting (e.g. "duplicate service billed", "weekly visit frequency", "distance between member and provider", "amount vs peer average", "shared contact info with provider", "weekend billing ratio", etc.). Column names should be self-explanatory; if a signal is 0 it means "not triggered" / "normal."

 **Important: the signals here are illustrative. Part of what we are evaluating is how you decide which signals matter, how you combine them into a meaningful assessment, and how you communicate that to a human investigator.**

That CSV is the **only** data input. You do **not** need to integrate with any external enterprise systems. You are free to enrich a case with **public** data sources (e.g. internet search, public business registries) if it makes your prototype more compelling — but it is not required.

4. What We're Looking For (Deliverables)

We are deliberately keeping this open-ended. At a minimum, we expect:

4.1 A working prototype

Something we can run locally — a web app, a notebook + lightweight UI, whatever you choose — that:

- Loads the 50 cases from the CSV.
- Produces an **AI-generated assessment per case** (summary + key signals + risk level + recommended next step).
- Lets a user **navigate the queue** and **drill into a single case**.
- Lets a user **interact with a case** in some meaningful way (ask a follow-up question, accept/reject an AI finding, leave a note — pick what feels most valuable).

You decide what "good enough" looks like for the prototype. Polish a small surface area well rather than half-building everything.

4.2 A short write-up (1–3 slides, PPT)

Tell us:

- **The product you imagined.** Who is the user, what is the core job-to-be-done, what did you choose to build vs. leave out, and why.

- **Your architecture.** How does the system reason about a case? Single LLM call? Multi-agent? Tool use? Retrieval? How do you keep it grounded and avoid hallucination? How would this scale beyond 50 cases?
- **Trade-offs.** What you would do differently with more time, what you deliberately cut, what risks you see (cost, latency, accuracy, trust, abuse, etc.).
- **How a human stays in the loop.** How do investigators trust, validate, and override the AI?

4.3 A short demo (optional but encouraged)

A 3–5 minute screen recording walking us through the prototype. Often more useful than written docs.

5. What We Care About

In rough priority order:

- **Product thinking.** Did you understand the user's actual problem and design something that *meaningfully* helps? Did you make crisp choices about what to include and what to leave out?
- **AI / system design.** Is your reasoning approach sensible? Is the AI grounded in the actual case data? How do you handle uncertainty, missing info, and bad outputs?
- **UX for the human-in-the-loop.** Is the AI's output presentable, scannable, and actionable for a busy investigator? Can they trust it?
- **Engineering quality.** Clean structure, runnable instructions, reasonable code. We don't need production-grade — we need *clear*.
- **Creativity.** Surprise us. If you have an idea that goes beyond the obvious "show a table of cases with a risk score," we want to see it.

What we are **not** grading on: pixel-perfect UI, framework choice, model choice, or feature checklists. There is no hidden right answer.

6. Constraints & Ground Rules

- **Use any stack, model, or framework you want** (OpenAI, Anthropic, open-source LLMs, local models, agent frameworks, no-framework, etc.).
- **Use any libraries / AI coding tools you want** (Copilot, Cursor, etc.). Just be ready to explain *your* design decisions.
- The CSV is **synthetic** — no privacy concerns, you can post snippets in prompts, logs, etc.
- **Do not** spend time building auth, multi-tenant infra, deployment pipelines, or anything that is not core to the AI experience.
- If you make assumptions about the business, **document them** in your write-up. We treat well-reasoned assumptions as a positive signal.

7. Submission

Please send us:

- A link to a Git repo (GitHub / GitLab / a zip is fine) containing your code.
- A README with **exact steps** to run the prototype locally (env vars, install commands, start commands, expected URL).
- Your write-up (markdown / PDF / Google Doc).
- Optional demo video link.

Good luck — we are excited to see what you build.